&

OPO STARTUPS · JULY 2026

AI-Powered Hybrid Search & Agentic Assistants

on Cloudflare's Edge

Build production search and durable AI agents that scale to zero — and back up.

Brett Bartrum

Founder · Juiced Wheels & ShopDAWG

Senior DevOps Engineer · Applied Systems

Why I Built This — It Starts at the Workbench

Before the architecture, the problem. At **Juiced Wheels** a bench tech has a customer's e-bike on the stand and the clock running — and the answer is buried in a parts catalog and a stack of manuals no keyword box can navigate.

84K+

parts to search

The right innertube is one SKU among tens of thousands across QBP · JBI · OSCO.

2,845

PDF manuals

The fix is on page 47 of a Bafang PDF — not in a tidy, queryable database.

"30h?"

cryptic error codes

Bafang, Bosch & Shimano each speak a different dialect — the code is meaningless without the table.

≠

symptom vs. keyword

"Motor cuts out climbing" isn't a phrase in any manual — it's a trace through the system.

A bench tech doesn't want ten documents — they want **the answer, cited, in seconds**, and bench time is a shop's biggest cost. That's the bar generic search couldn't clear, so I built **Shop Shepherd** on Cloudflare for ~$80/mo — and the stack underneath **isn't bike-specific.**

The Problem with Traditional Search Infrastructure

⚠ EVEN WHILE YOU READ THIS SLIDE…

$0.0000

Burned by always-on infra

($1,950/mo baseline, $0.0007/sec)

0

Queries actually served

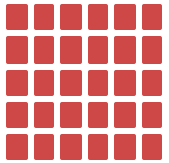
100%

Of spend on idle capacity

Elasticsearch / OpenSearch

Dedicated clusters running 24/7 even with zero traffic.

$200–800+/mo minimum, no queries needed



Pinecone / Weaviate

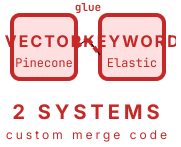
Vector DB subscription bills for **all 30 days**, used or not.

$70–250+/mo serverless tier

Regional Lock-in

Pinecone, ES clusters pin to one region. EU/Asia queries pay **200ms+** tax.

Multi-region · 2–3× the cost or self-managed replication



Stitched Stack

Vector DB + keyword search are **separate systems**. Two bills, two query APIs, glue code to merge.

No native hybrid · sync drift · double the ops

Why Cloudflare? The Edge Advantage

✓ SAME SLIDE, ON CLOUDFLARE'S EDGE…

$0.0000 —

Always-on cost

(flatline — pay only for what runs)

1,247,832

Queries served while you read

$24.96

Total spend

(~$0.00002 / query)

Usage-Based Pricing

Pay per request, per row, per query — not per hour.

Zero traffic = zero cost

Global Edge Network

Workers run within ms of the user, not in us-east-1.

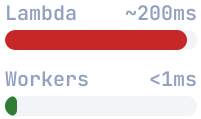
330+ PoPs worldwide

Unified Platform

Workers + D1 + Vectorize + KV + R2 in one ecosystem.

One bill, no glue code

Zero Cold Starts

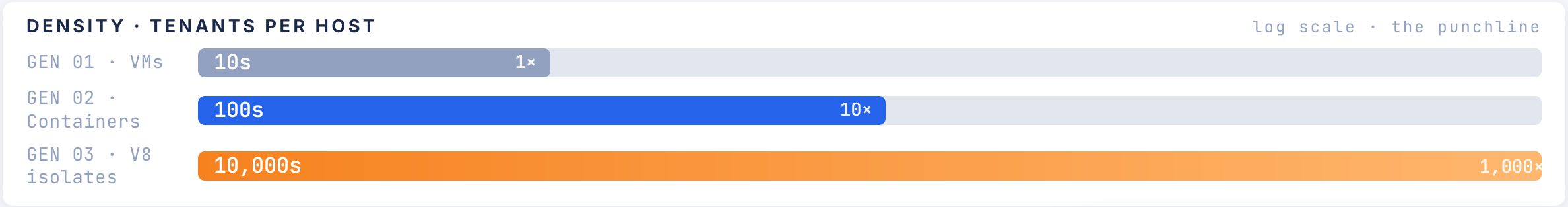


V8 isolates start in sub-ms — the shop lights flick on, no warming a cold engine.

200× faster than containerized FaaS

Three Generations of Compute

Each one cut cost and blast radius by an order of magnitude. V8 isolates are what makes scale-to-zero affordable at edge density.



GEN 01 · ~2001 — VMware · KVM · EC2

Virtual machines

Full guest OS · hypervisor

Footprint	GB
Cold start	30–120 s
Density / host	10s
Idle cost	100%

Bare-metal era. Pay 24/7 for capacity you may never use.

GEN 02 · ~2013 — Docker · Lambda · Fargate

Containers

Shared kernel · namespaced procs

Footprint	100s MB
Cold start	200–800 ms
Density / host	100s
Idle cost	scale-to-zero*

**Then a 500 ms cold start the next time someone shows up.*

WHAT I USE

GEN 03 · ~2017 — Cloudflare Workers · Deno Deploy · Fastly

V8 isolates

One process · thousands of tenants

Footprint	~3 MB
Cold start	<5 ms
Density / host	10,000s
Idle cost	$0 · truly

Same V8 sandbox Chrome uses for tabs — battle-tested at billions of executions/day.

PART ONE

Search Architecture Deep Dive

Three layers of search, one edge platform

$ The Search Stack — Think of It Like Shopping

Three search engines, three shopping behaviors. The customer's voice picks the station: semantic for symptoms, faceted for browsing with constraints, keyword for exact parts.

FIND THE RIGHT AISLE

Vectorize SEMANTIC

> *"My bike has a flat tire"*

`768d cosine · topK=10 · ~15 ms†`

PICK OFF THE SHELF

D1 + Meilisearch FACETED

> *"Innertube — 26" × 4", Schrader valve"*

`facets · typo-tolerant · ~28 ms‡`

SCAN THE BARCODE

D1 + FTS5 KEYWORD

> *"I need part TU1200"*

`SQL · BM25 · ~3–11 ms`

Measured engine times, warm queries. †Semantic embed: ~3ms cached / ~200ms cold. ‡Meili is remote-hosted (NYC); warm round-trip adds ~400–700ms — FTS5 + Vectorize are co-located in the worker.

~ Layer 1: Find the Right Aisle — Vectorize

Why Vector Search Matters

Keywords fail when users describe problems instead of naming parts. "My bike has a flat tire" should match "innertube replacement" — vector search makes this connection.

Workers AI maps each sentence to **768 numbers** — coordinates on a "meaning map," where similar meanings land near each other even with zero shared words. `embeddinggemma-300m` generates them; Vectorize finds the nearest via cosine similarity.

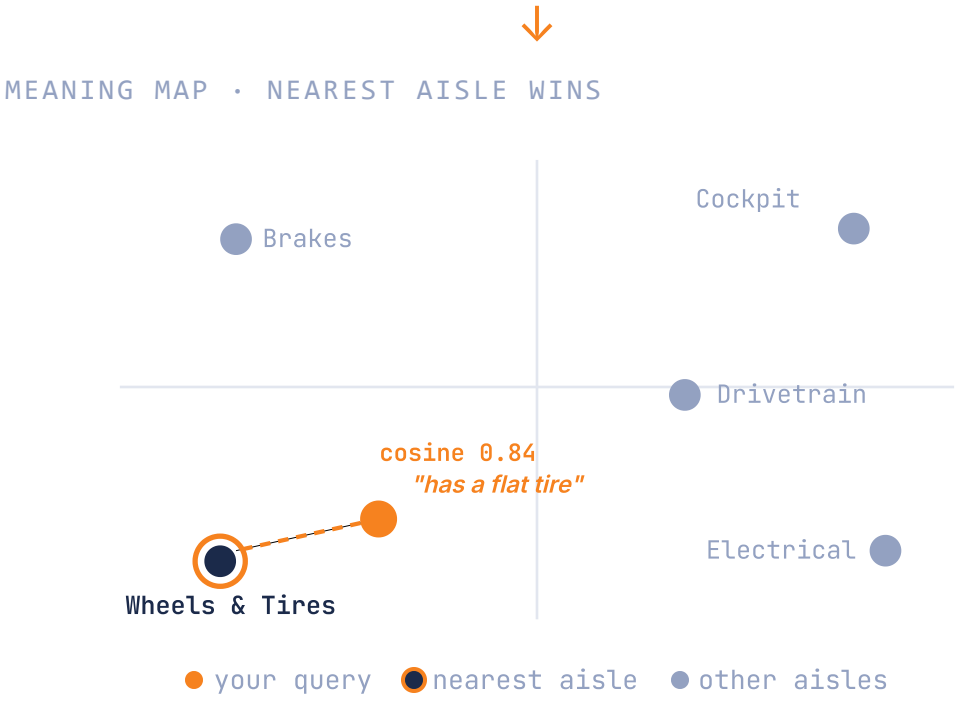
Embedding Pipeline

1. Auto-chunk content > 7,500 chars
2. Workers AI → 768-dim vector
3. Vectorize index → stored at edge
4. KV cache → skip re-embedding

Semantic Routing

CUSTOMER ASKS

"My bike has a flat tire"



VECTORIZE STAGES 1 embed (Workers AI) → 2 cosine search → 3 topK=10

Layer 2: Pick Off the Shelf — D1 + Meilisearch

What Faceting Is

Filtering a big catalog by structured attributes — brand, wheel size, valve type, price, in-stock — with a **live count on every option**. It's how a tech narrows 84,000 parts down to the 8 that fit, without typing a perfect query.

Why D1 *and* Meilisearch?

D1 system of record

Normalizes 84K parts from 3 vendor feeds, scores e-bike relevance, and serves keyword search via FTS5 — the fallback if Meili is ever down.

Meilisearch the speed layer

A sub-30ms count per facet checkbox across all 84K rows — more than a D1 `GROUP BY` can handle — plus typo tolerance ("shrader" finds "Schrader"). It's a faceted-search engine with its own hybrid keyword + vector ranking.

Faceted Shelf in Action

CUSTOMER ASKS

"Innertube — 26" × 4", Schrader valve"

FILTERS APPLIED

☑ Tire width: 4.0"	84
☑ Wheel diameter: 26"	46
☑ Valve type: Schrader	24
☑ In stock	8

26" × 4.0–4.8" Fat Innertube
Kenda · 35mm Schrader valve · $13.99

typo-tolerant · ~28ms · fallback to D1 FTS5

THE LONE THIRD-PARTY

Meilisearch ($30/mo) is the only non-Cloudflare piece of this whole stack — today's one platform gap for fast faceting + typo tolerance. If Cloudflare ships a native equivalent, this line drops to **$0** and the architecture is 100% first-party.

MEILISEARCH STAGES `1 parse facets` → `2 filter (typo-tolerant)` → `3 rank by relevance`

Layer 3: Scan the Barcode — D1 + FTS5

How It Works

D1 is Cloudflare's serverless SQLite; FTS5 is its built-in full-text search — co-located with your Worker, no extra service, no extra bill.

FINDING · PROGRESSIVE RELAXATION — one round trip

EXACT	"TU1200" → 1 hit
PREFIX	"TU120…" → 6 hits
FUZZY	"TU~1200" → 18 hits

RANKING · HOW BM25 SCORES A HIT — 3 signals → one score

RARITY	"TU1200" outweighs a common "tube"
FREQUENCY	more hits help — but less each time
BREVITY	a title hit beats one deep in a manual

Exact-Match Lookup

CUSTOMER ASKS

"I need part TU1200"

TU1200

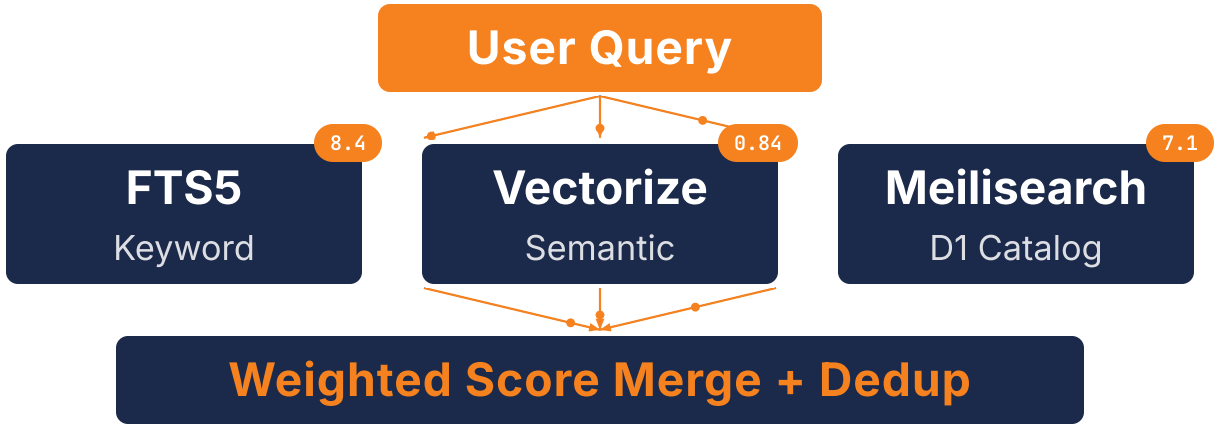
Fat Bike Tube · 26×4 Schrader

BM25 score 8.4 · 7 ms (measured)

FTS5 STAGES: 1 tokenize → 2 match (virtual table) → 3 rank (BM25)

Hybrid Search: Merging Keyword + Vector

When one engine isn't enough, fire all three in parallel and merge by weighted relevance score. The agent never sees three result lists — just one ranked output.



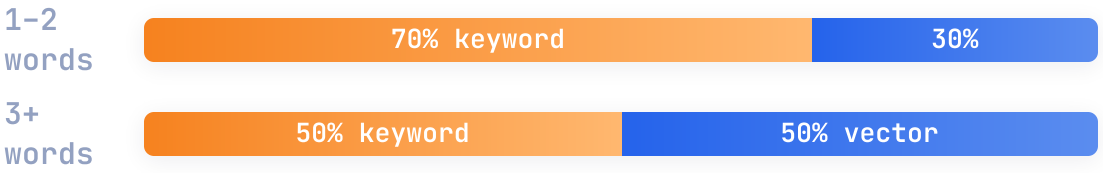
TOP RESULT | TU1200 Fat Bike Tube — matched by all three engines — final 18.2

Source Weights

Manuals 2.0× · Docs 1.5× · Parts 0.2×

- OEM manuals win every tie — they're the authoritative source for technicians
- Parts get demoted unless the query is clearly a SKU lookup
- A result returned by multiple engines stacks the boosts

Short Query Optimization

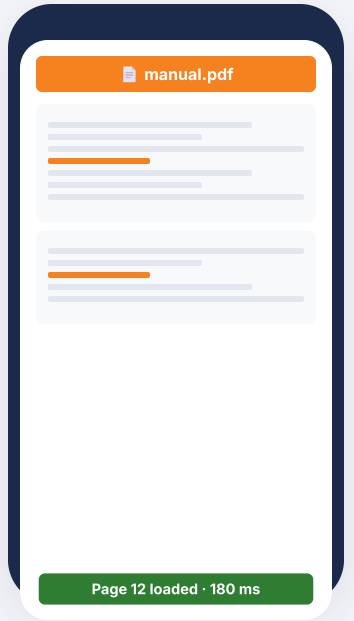


- Short queries lean keyword — prevents false matches like *"venue"* ≠ *"venus"*
- Below-floor results get cut, with a top-3 fallback so the agent never starves

PDF Pipeline: OCR, Extract, Optimize

Why OCR Every PDF?

Most OEM service manuals are scanned images or flattened exports — there's no selectable text layer. **OCR (Optical Character Recognition)** turns those page images into searchable text. I then inject that text **back into the PDF as a hidden layer**, so the manual is fully searchable in any viewer while looking identical to the original. **You can't chunk what you can't read.**



Tech reads at the bench

RAW OEM PDF → MOBILE-READY

40× smaller · same content · ready for a 4G phone at the bench

HOW I GET THERE

- ✓ Subset embedded fonts *(strip unused glyphs)*
- ✓ Recompress images *(JPEG2000 / lossy DCT)*
- ✓ Inject OCR text layer *(hidden, selectable, indexable)*
- ✓ Optimize via Adobe to PDF/A-2u *(latest archival standard)*
- ✓ Web-linearize *(first page renders before full download)*

PDF STAGES `1 OCR` → `2 chunk + embed` → `3 D1 + Vectorize` → `4 R2 (mobile + thumbs)`

Why Chunking Context Matters

The Naive Chunking Problem

Most tutorials split documents into fixed-size blocks and embed them as-is — like cutting a manual into unlabeled index cards. You get a match back, but no idea which card it's from.

An orphaned chunk like *"torque the bolts to 40 Nm"* is useless without knowing it's a Bosch CX mid-drive install vs. a Shimano hub motor teardown.

Long manual PDF · 142 pages — Bosch CX Mid-Drive Install Guide

↓ chunked + tagged

chunk_0042
"Torque bolts to 40 Nm. Check alignment..."
bosch-cx install

chunk_0043
"Speed sensor mounting: 5 Nm. Use anti-seize..."
bosch-cx sensor

chunk_0044
"Battery seat alignment: shim to 0.2 mm..."
bosch-cx battery

Naive Chunk (no context)

```
{
  text: "Torque bolts to 40 Nm.
        Check alignment before
        tightening...",
  vector: [0.23, -0.41, ...]
}
```

Metadata-Tagged Chunk (structural context)

```
{
  text: "Torque bolts to 40 Nm...",
  vector: [0.23, -0.41, ...],
  metadata: {
    title: "Bosch CX Installation",
    path:  "/guides/bosch-cx-install",
    tags:  ["mid-drive","torque"],
    category: "installation",
    source: "manual"
  }
}
```

NEXT **Metadata fixes naming.** But what about OCR garbage where even the chunk text is scrambled? →

Contextual Retrieval: Quality Up, Cost Down

1 CHUNK TEXT *raw bytes* + 2 METADATA *structural · prev slide* + 3 **context_llm** *semantic · this slide*

Metadata gives chunks a structural home. A 1–2 sentence **LLM summary** — Anthropic's Contextual Retrieval pattern — adds a sticky note that stays findable even when the page is OCR garbage.
Labor division: *LLMs author chunk context; humans author the graph (slides 19–21) — enrich text, don't invent ontology.*

BEFORE — RAW OCR CHUNK

Indexes can't find it

```
"340 % 8 1 240% TURBO 340%
3 4 0 SPEED SENSOR BDU
4xx INSTALL torque 5Nm..."
```

- **FTS5 BM25:** 0 useful matches — query terms don't appear in garbled OCR
- **Vectorize:** embedding of garbage = garbage similarity

Query: "Bosch Drive Unit speed sensor installation"

AFTER — CHUNK + context_llm

Both indexes hit at rank 1

> context_llm
> *Bosch DriveUnit BDU4xx (MY24) installation guide — speed sensor mounting and torque spec.*

```
"340 % 8 1 240% TURBO 340%..."
```

- **FTS5 BM25:** "Bosch", "DriveUnit", "speed sensor" all match the summary
- **Vectorize:** coherent summary embeds cleanly → high cosine sim

RETRIEVAL QUALITY · n=30 cases

Top-1 accuracy

66.7% → **93.3%** +26.6 pp

Top-3 accuracy

83.3% → **100%** +16.7 pp

COST PER TURN

Dollars per turn

$0.201 → **$0.075** −63%

Step-2 input tokens

49.6K → **13K** −73%

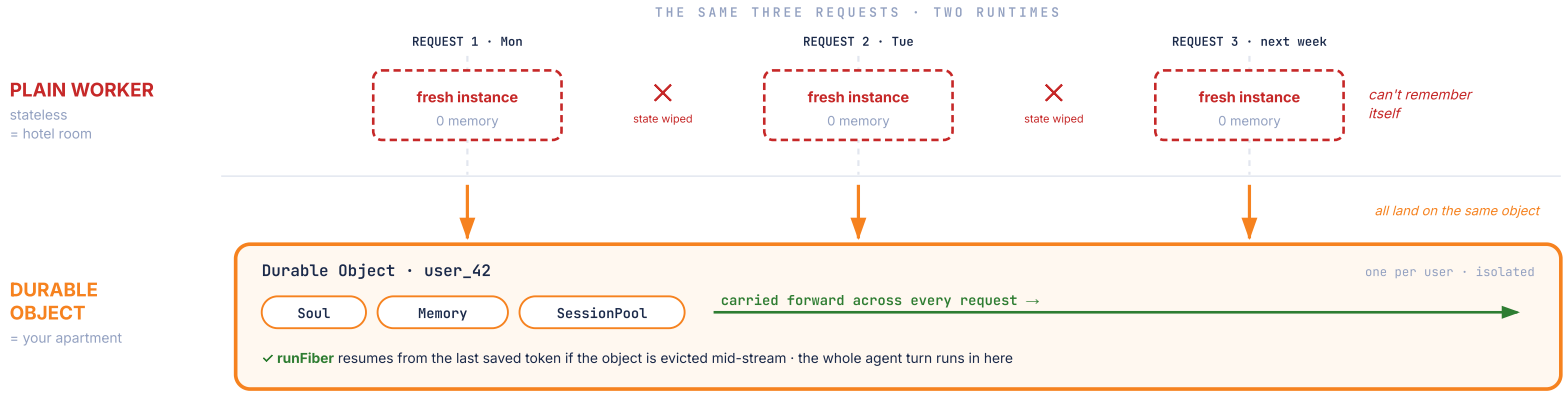
PART TWO

From RAG to Agentic AI

Everything so far was stateless — every request started cold. Now I add what search can't have: durable per-user memory, crash recovery, and query-aware context compression. Watch one number the whole way down: a tool turn drops from ~36¢ to ~3¢.

A Durable Object Per User (Project Think)

A Worker is a **hotel room** — fresh every request, your stuff is gone. A Durable Object is your **apartment** — same address, same key, your memory is still on the counter.



TOOL	PURPOSE
`search_documentation`	unified hybrid search
`search_manuals`	PDF manual sections
`search_parts`	faceted parts catalog
`diagnose`	symptom → ranked causes (graph)
`read_manual`	deep-read one manual
`get_content_section`	fetch a known section
`search_internet`	web fallback — recalls, news

PER-USER DURABLE OBJECT STATE

Soul block
System prompt from KV — live-refresh per turn

Memory block
Per-user facts via UserMemoryProvider — durable identity

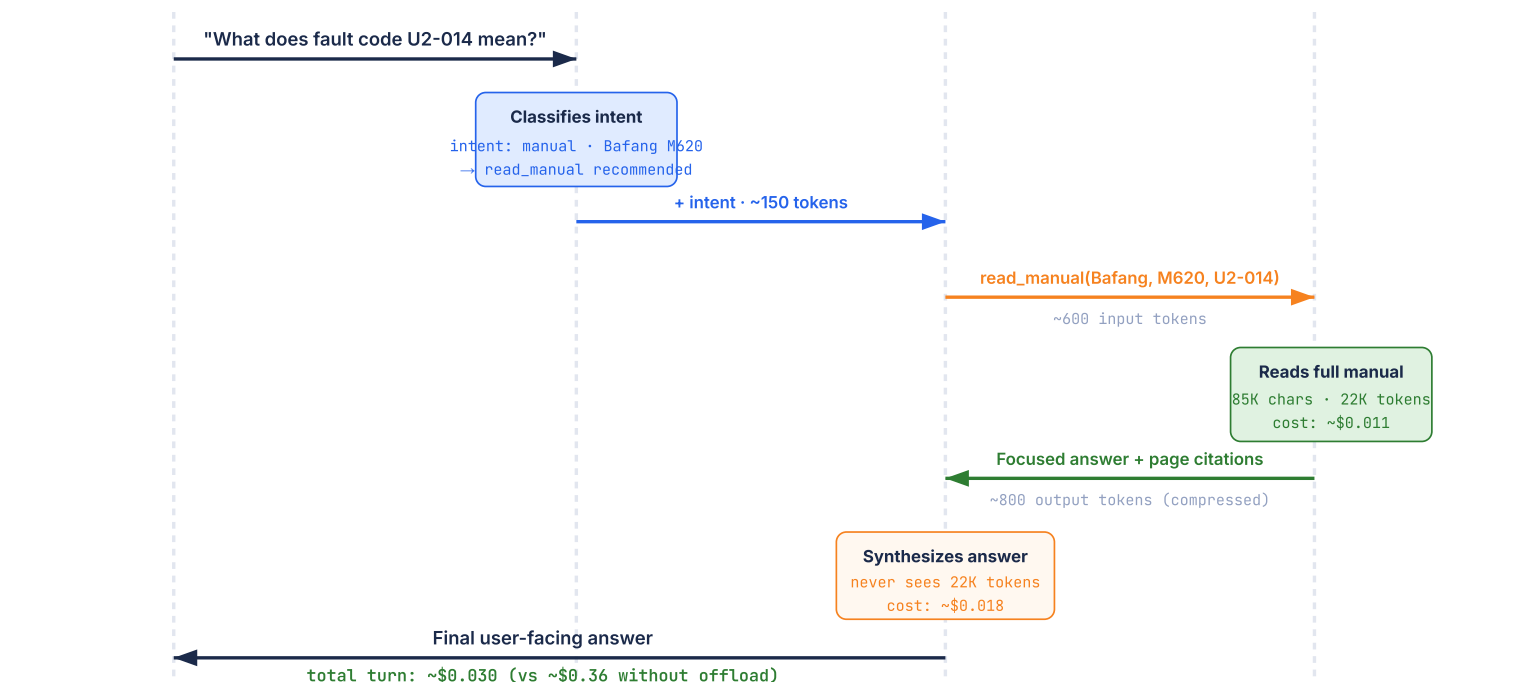
SessionPool
Per-thread history, FTS5-searchable

chatRecovery
runFiber — mid-crash saves partial text

Sub-Agent Offloading: Cheap Bulk Work, Premium Synthesis

High-token tool work goes to a near-free model; the premium model only sees the compressed result and writes the answer. Like a shop, the apprentice does the bulk grunt work and the master tech makes the call.

~$0.36 → ~$0.030 per turn — a 90% cut, same answer quality.



THROUGHPUT
TOKENS / SECOND

Cerebras gpt-oss-120b *(used here)*

~3,000 t/s — 20–60× faster

Groq · Together *(specialized inference)*

~500–1,000 t/s — ~7×

Sonnet 4.6 · GPT-5 · Gemini 2.5 *(frontier APIs)*

~50–150 t/s — 1× baseline

20–50× faster than frontier APIs.
The pre-agent adds *~250ms*, not *~3 seconds* — that's what makes the offload pattern viable.

Why Cerebras for the mini-agent: I trialed Claude Haiku and Cloudflare Workers AI models too — both viable, but Cerebras's speed at low cost was the best fit for work that runs on every turn.

Query-Aware Compression: 90% Cheaper, 10× Sharper

UNIT ECONOMICS

× BEFORE · DUMB TRUNCATION

The model gets noise.

Like clipping the manual at page 16.

- Tool returns **15 KB** raw PDF text
- Char-count clip at 16K characters
- Sonnet sees 8 irrelevant pages of OEM filler
- Burns **8 follow-up tool calls** narrowing down
- No page citations, no section anchors

Response: 230 chars of vague advice

✓ AFTER · QUERY-AWARE COMPRESSION

The model gets answers.

Like a service writer marks the right page.

- Same 15 KB result — pre-Sonnet hop adds `gpt-oss-120b`
- Cerebras sub-agent extracts query-relevant sections
- **Page citations + chunk IDs preserved**
- Sonnet receives **3 KB** of focused content
- Answers in **2 calls** with diagnostic depth

Response: 2,239 chars of specific guidance

INPUT TOKENS — -80%

~~15K~~ → 3K

per tool result sent to Sonnet

TOOL CALLS / ANSWER — -75%

~~8~~ → 2

no follow-up narrowing

COST / TURN — -90%

~~$0.36~~ → $0.03

≈ $330 saved / 1K turns

USEFUL RESPONSE — +10×

~~230~~ → 2,239

chars of specific guidance

RAG retrieves. Prompting reasons. Compression curates.

Always-compress tools: `read_manual` · `get_manual_content` · `get_content_section` · others compress only above the 4K-token budget

When retrieval *isn't enough*.

Search returns **text**. Diagnosis needs **causality** — what supplies what, what cuts what, what symptom traces to which subsystem.

01 WHAT I HAD

Hybrid search over 2,845 manuals, 2,334 articles, 84K parts.

- FTS5 + Vectorize + Meilisearch fused via `/api/search/unified`
- Weighted merge + ***Gemini structural rerank*** — ranked passages with page citations

*Great at finding the **page**. Useless at finding the **cause**.*

02 WHAT BROKE

"Motor cuts out climbing" returned 12 pages.

- Pages described the ***symptom*** (motor cutting out under load)
- None named the ***cause*** (controller overheating) or the ***subsystem*** (thermal cutoff disabling motor power)

*A symptom isn't a keyword. It's a **trace** through the system.*

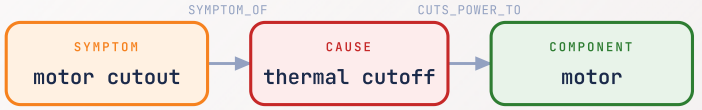
03 WHAT I BUILT

A diagnostic graph built on physics, real-world conditions & rider behavior.

- **825 profiled nodes** — ***each carries tools · symptoms · tiered $0-first steps***
- **2,523 typed edges** — causal verbs the agent calls per turn via `query_ebike_graph`

`SUPPLIES_POWER_TO · CUTS_POWER_TO · SYMPTOM_OF · SENDS_SIGNAL_TO · TESTED_BY`

ONE CAUSAL TRACE, WALKED PER QUERY



NEXT THREE SLIDES I tried LLM extraction. It hallucinated. I shipped hand-crafted instead — and the agent still queries the graph every diagnostic turn. Here's the trade-off, the anatomy, and the live trace.

Why *hand-crafted* beat LLM extraction.

LLM EXTRACTION · TRIED IT

DEAD-END

High cost. High risk.

- **Hallucinated edges** (`motor PART_OF tire?`) → fabricated repair advice
- 963 docs → `Anthropic Batch API` → entities + relationships
- 31 duplicate "Battery" variants → canonicalization nightmare
- No auto-sync — graph drifts every time a doc changes
- ~$40 per full re-extraction × every doc update
- No audit trail — can't diff a node's history

Kept it OFF in production

HAND-CRAFTED · SHIPPED IT

SHIPPED

$0 cost. 100% deterministic.

- 825 nodes in TypeScript — one file, one source of truth
- Expert-authored with e-bike diagnostic ordering
- Regenerate SQL + seed in seconds — zero LLM cost
- Zero hallucinated edges → no fabricated repair advice
- Every change is a PR with a reviewable diff
- Agent still uses GraphRAG to `traverse` — humans author, LLMs query

LIVE in production, every diagnostic query

> "**Truth is human work.** Reasoning over it is LLM work."

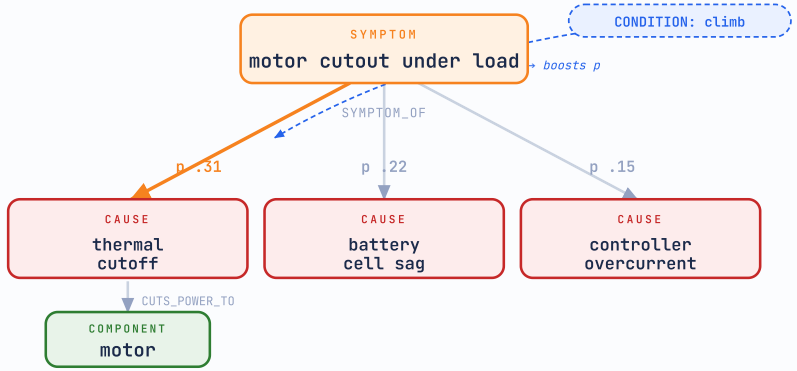
INTEGRATED WITH EVERY DIAGNOSTIC QUERY

```
diagnose('motor cutout climbing') → sym_motor_cutout_under_load (12 ms · D1)
query_ebike_graph('controller', 'interrupts') → brake_cutoff_sensor, shift_sensor (safety overrides)
lookup_error_code('E021') → controller_thermal_cutoff (69 codes indexed)
```

Not a search index — a diagnostic brain.

825 expert-authored diagnostic profiles · **2,523** weighted causal edges — *error codes are just one way in*.

ONE DIAGNOSTIC SLICE · D1 + KV



Think of it as the bike's wiring diagram the AI can walk: parts are the **nodes**, the labeled wires between them (**typed, weighted edges** — SYMPTOM_OF · CUTS_POWER_TO) are the links, and the agent *traces the circuit* from symptom to cause instead of reading a paragraph that happens to mention the part. A real slice of the 825-node graph; live **conditions** (a climb) re-weight the paths. **Expert-authored, not LLM-scraped.**

☐ symptom ☐ cause / failure ☐ component edge = relation · p = probability

ALSO IN THE GRAPH

18 node types · 85 typed relations · environmental multipliers (rain · cold · cargo)

`SYMPTOM_OF · CAUSES_FAULT_IN · FAILS_AS · TESTED_BY · INTERRUPTS · CUTS_POWER_TO`

A PLAYBOOK PER NODE — `thermal_cutoff`

TOOLS	multimeter · IR thermometer · torque wrench
CONSUMABLES	dielectric grease · thermal paste
SYMPTOMS	cutout under load · power returns after cooldown
TIER 0 · $0	temp check after a climb · airflow + brake drag
TIER 1 · meter	temp-sensor resistance · connector continuity
TIER 2 · swap	replace thermal sensor / controller

Every node carries this — cost-tiered, e-bike-specific, cheapest-first.

HOW GRAPHRAG ANSWERS — *"motor cuts out climbing"*

1. **MATCH** — `diagnose` maps the phrase to a **node** — codes & the 768D vector are other ways in → `sym_motor_cutout_under_load`
2. **TRAVERSE** — `query_ebike_graph` walks causal edges to pull the connected subsystem
3. **RANK** — scored by base p × live conditions (climb = load + heat) → `thermal_cutoff · battery_cell_sag · controller_overcurrent`
4. **ANSWER** — Sonnet reasons over the **causal subgraph** and names the cause + the fix

Plain RAG retrieves *passages*.
GraphRAG retrieves the **causal chain**.

LIVE DEMO THE AGENT IN MOTION

From symptom to *diagram* — live.

Shop Shepherd · shepherd.shopdawg.app claude-sonnet-4-6

Customer says the motor cuts out when climbing hills. Bafang M620.

→ tool: diagnose

```
{ "query": "motor cutout climbing", "limit": 3 }
```

← sym_motor_cutout_under_load 12 ms · D1

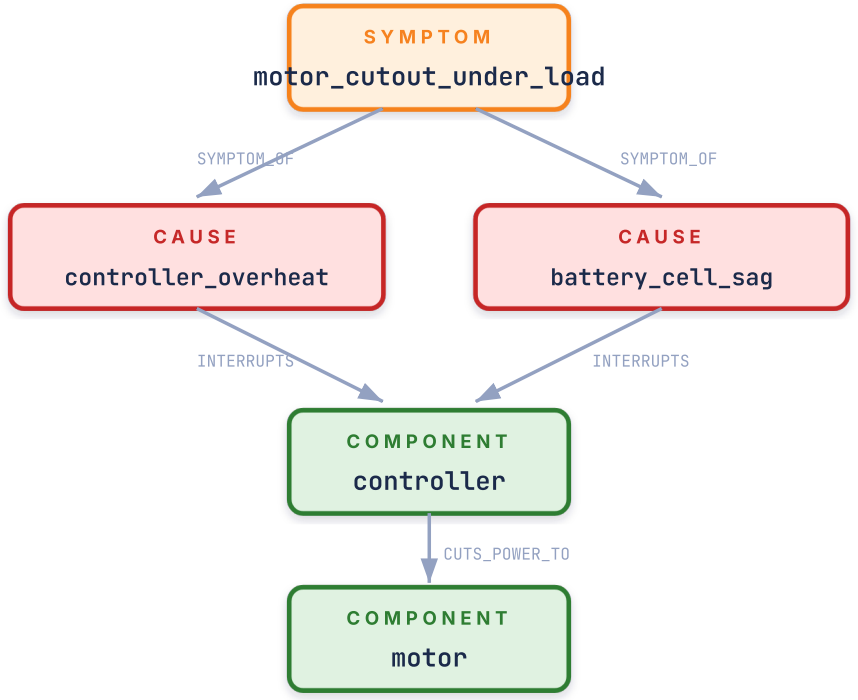
→ tool: query_ebike_graph

```
{ "node": "controller", "verb": "interrupts" }
```

← 4 nodes, 5 edges · rendering trace… 8 ms · D1

Likely **controller overheating** or **battery cell sag** — under sustained climbing load, pack voltage drops or the controller overheats, and its cutoff trips. Diagram on the right.

GENERATED MERMAID · RENDERED INLINE graph TD



symptom cause component → LIVE DEMO

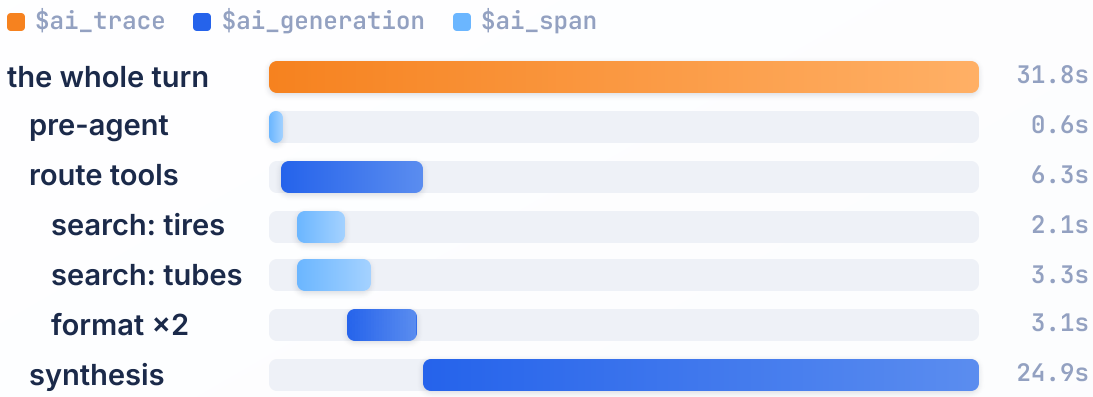
IF YOU DON'T WORK ON BIKES **motor** = the web server · **controller overheating** = CPU throttling under load · **battery sag** = DB / resource latency under load

Observability: One Pane of Glass

I was already running PostHog for product analytics — funnels, retention, session replay. Turning on its **LLM observability** dropped every AI trace onto that same dashboard: cost, tokens, latency, and tool-routing, per turn.

ONE REAL TRACE · POSTHOG

"Best margin on 26×4 fat tires + tubes, grouped by distributor"



44.9K tokens · $0.072 · 8 spans. The Sonnet **synthesis alone is 24.9s — 78% of the turn**; the 3 searches + 2 Cerebras formatters total ~3s and $0.009. You only see that once it's traced.

ONE PANE OF GLASS

AI traces land right next to the product analytics I already had.

Per-turn cost & tokens — cache-aware, so prompt-cache savings show up

Tool routing — which tools fired, in what order, how long each took

Latency per step — pre-agent · tools · synthesis

Beside **funnels, retention & session replay** — already running

Recommended, not required — and **not part of the ~$80/mo search infra**. PostHog was already in the stack; the LLM layer was icing on the cake.

posthog.com/docs/ai-observability

Real-World Scale: What This Handles

2,845

PDF Manuals Indexed

Gathered with AI search — Brave · Tavily · Firecrawl

OCR'd · ~593K chunks · full-text + vector indexed

2,334

Repair Articles

Built from Google Deep Research, human-curated

full-text + vector indexed

84K+

Parts in Catalog

Nightly refresh from 3 vendor feeds

D1 faceted, Meili-accelerated

$80/mo

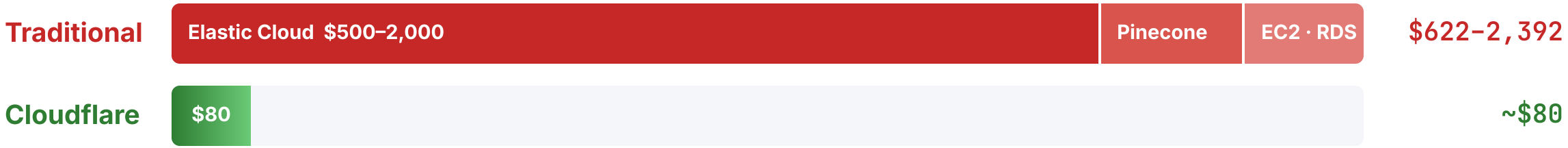
Production Stack Cost

Workers + D1 + Vectorize + Meili included

$ Cost Comparison: Traditional vs. Cloudflare

UNIT ECONOMICS

WHERE THE MONTHLY SPEND GOES · REPRESENTATIVE PRODUCTION SCALE · JUN 2026 RATES



CLOUDFLARE'S $80, ZOOMED IN 25× — WHERE IT ACTUALLY GOES

Orange = Meilisearch, the only paid third-party — skip it and D1 FTS5 covers parts (Workers Paid $5 · D1 free tier).

25× cheaper · same throughput · scale-to-zero baseline

25×	$0.5–2.3K	$6.5–28K	$0
CHEAPER THAN THE TRADITIONAL STACK	SAVED EVERY MONTH	SAVED EVERY YEAR	WHEN IDLE · SCALE-TO-ZERO

Lessons from Production

Five hard-won lessons from running this in production — **not the order I learned them, but what I'd hand my past self on day one.**

1

OBSERVABILITY

Observability first

Instrument before you optimize with PostHog — you can't tune what you can't measure.

2

COST

Offload heavy work

Cheap models do bulk context; a premium model synthesizes. ~10× cheaper.

3

JUDGMENT

Hand-author the graph

Truth is human work; let the LLM traverse it. 825 profiled nodes beat batch-API extraction.

4

METADATA

Contextualize chunks

Tag every chunk: source, section, vehicle class, date. Context disambiguates.

5

TESTING · CI

E2E over mocks

Live end-to-end tests on every deploy catch what unit mocks never will.

THE ONE-LINE VERSION

Let **cheap models do the volume**, keep **humans on the judgment calls**, and **measure everything** in between.

PART THREE

Build It Yourself

Two steps, not two choices — ship on AI Search now, graduate to the primitives when you scale.

None of it is bike-specific — swap in your industry's manuals, parts & diagnostics and the architecture holds.

STEP 1 · THE SHORTCUT START HERE

1 · AI SEARCH — SHIP THIS WEEKEND → 2 · CUSTOM STACK — WHEN YOU SCALE

Cloudflare *AI Search*.

The same architecture you just saw — provisioned for you in a few clicks.

WHERE AI SEARCH FITS IN YOUR STACK

~80% of this deck without writing a Worker. It provisions the whole **retrieval pipeline** you just built:

✓ AI SEARCH

Hybrid RAG — vector + BM25 keyword, reranked

R2 · Workers AI · Vectorize · AI Gateway · query rewriting · cited answers

Faceted parts Meilisearch · typo-tolerant filters

Agent + Memory Durable Objects · diagnostic graph

↑ bolt these on in Step 2, as you outgrow the shortcut

IF YOU REMEMBER ONE THING

Same R2 + Workers AI + Vectorize + AI Gateway pipeline.

Click to provision. No wrangler glue.

Drop docs into a bucket. Auto-chunked, auto-reindexed.

Hit one endpoint. Ranked results or full RAG answer.

Same scale-to-zero economics. Same edge latency.

developers.cloudflare.com/ai-search

Cloudflare AI Search is in **open beta** — free within limits, 4 MB max per file.

STEP 2 · THE LONG ROAD WHEN YOU OUTGROW THE SHORTCUT

1 · AI SEARCH — SHIP THIS WEEKEND → 2 · CUSTOM STACK — WHEN YOU SCALE

Same primitives, *full control*.

Started on **AI Search**? Steps 1–4 (the whole hybrid retrieval pipeline) come provisioned — your real build is the **agent + memory + domain graph**.

1

DATABASE · SQL

D1 + FTS5 for keyword ✓ via AI Search

```
wrangler d1 create my-search-db
CREATE VIRTUAL TABLE ... USING fts5()
```

2

VECTORS

Vectorize for semantic ✓ via AI Search

```
wrangler vectorize create my-index \
--dimensions=768 --metric=cosine
```

3

INGEST · COMPUTE

Worker to ingest & embed ✓ via AI Search

Chunk with metadata, embed via Workers AI, write to D1 + Vectorize in one transaction.

4

SEARCH · API

Hybrid endpoint ✓ via AI Search

Parallel FTS5 + Vectorize, weighted merge by source, threshold + dedup.

5

AGENT · MEMORY

Agent layer for tools & memory

Install `@cloudflare/think` — streaming, tools, per-user Durable Object memory, crash recovery.

GRADUATE TO STEP 2 WHEN ANY OF THESE IS TRUE

- **Big docs** — PDFs past the beta's ~4 MB file ceiling
- **Per-user state** — durable memory + crash recovery (Durable Objects)
- **Custom scoring** — hand-tuned hybrid ranking / your own rerankers
- **Causal diagnosis** — typed-edge GraphRAG, not semantic proximity
- **Faceted catalog** — typo-tolerant facet filters over 84K parts (Meilisearch)
- **Tool breadth** — a long tail of bespoke agent tools

&

Start Building Today

Everything shown here runs on Cloudflare's free or pay-as-you-go tiers

Start on AI Search → graduate to the primitives as you grow. Same edge, same economics, no rip-and-replace.

- developers.cloudflare.com/d1
- developers.cloudflare.com/vectorize
- developers.cloudflare.com/workers-ai
- cloudflare.com/startups
- blog: AI Search primitive
- blog: Project Think

See it live → shepherd.shopdawg.app

SCAN FOR THESE SLIDES

Built for e-bikes. The primitives fit any industry.

Same engine, healthcare next — point these primitives at your domain.